

Set之二



定义自己的HashSet

北京理工大学计算机学院
金旭亮

自定义HashSet

为了深刻理解HashSet的基本原理，我们尝试编写一个自定义的HashSet，为了避免事情变得过于复杂，让相关的编程技巧变得更简洁易懂，我们使用二维数组而不是HashMap来保存集合元素：

MyHashSet		
m	insert(E)	void
m	contains(E)	boolean
m	storeLocation(E)	int
m	toString()	String
p	capacity	int

```
public class MyHashSet<E> {  
    //此HashSet最多可以保存10个元素  
    private static final int STORE_SIZE = 10;  
    //每个槽最多个保存4个元素 (这些元素的hash值相同)  
    private static final int SLOT_SIZE=4;  
    //使用二维数组来保存元素  
    private Object[][] store = new Object[STORE_SIZE][SLOT_SIZE];  
  
    //获取集合的容量  
    public int getCapacity(){  
        return STORE_SIZE;  
    }  
}
```

向自定义HashSet中插入元素

//计算hash值, 它决定了真实的存储位置

```
private int storeLocation(E elem) {  
    return Math.abs(elem.hashCode()) % STORE_SIZE;  
}
```

在向集合中插入元素时, 通过它的hash值确定保存在二维数组中的哪一行, 之后, 再在这行中寻找空位, 将其保存到这个空位中。
如果没有空位了, 就抛出异常。
如果发现已有同样值的元素了, 直接返回。

//插入元素

```
public boolean insert(E elem) throws Exception {  
    Object[] target = store[storeLocation(elem)];  
    int idx = 0;  
    //在行中查找“空位”  
    while (target[idx] != null && idx < SLOT_SIZE) {  
        //如果已有这个元素, 则不追加它到集合中  
        if(target[idx].equals(elem)){  
            return false;  
        }  
        idx++;  
    }  
    if (idx == SLOT_SIZE) {  
        throw new Exception("槽已经满了");  
    }  
    target[idx] = elem;  
    return true;  
}
```

查询元素

```
//是否已经包容有特定的元素
public boolean contains(E elem) {
    //计算Hash值, 按照此值找到存储的行
    Object[] target = store[storeLocation(elem)];
    //在行中查找是否此元素已经存在
    int idx = 0;
    while (target[idx] != null) {
        if (elem.equals(target[idx])) {
            return true;
        }
        idx++;
    }
    return false;
}
```

调用equals()方法实现判等

通过遍历生成字符串

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    for (Object[] row : store) {
        int idx = 0;
        while (row[idx] != null && idx < SLOT_SIZE) {
            sb.append(row[idx]).append(", ");
            idx++;
        }
    }
    if (sb.length() >= 2) {
        sb.setLength(sb.length() - 2);
    }
    return sb.toString();
}
```

使用示例

Student	
m equals(Object)	boolean
m hashCode()	int
m toString()	String
p name	String
p gpa	float

Student类重写了equals和hashCode方法，使用两个字段实现判等操作。

```
public static void main(String[] args)
    throws Exception {
    var myHashSet = new MyHashSet<Student>();
    Student stu = null;
    for (int i = 0; i < 5; i++) {
        //创建一个示例元素
        stu = createStudentObject();
        myHashSet.insert(stu);
        //相同元素追加两次，第二次不会成功
        myHashSet.insert(stu);
    }
    System.out.println(myHashSet);
    //返回true
    System.out.println(myHashSet.contains(stu));
}
```

UseMyHashSet.java